

PAQRAP · SUSTENTACIÓN FINAL DEL PROYECTO

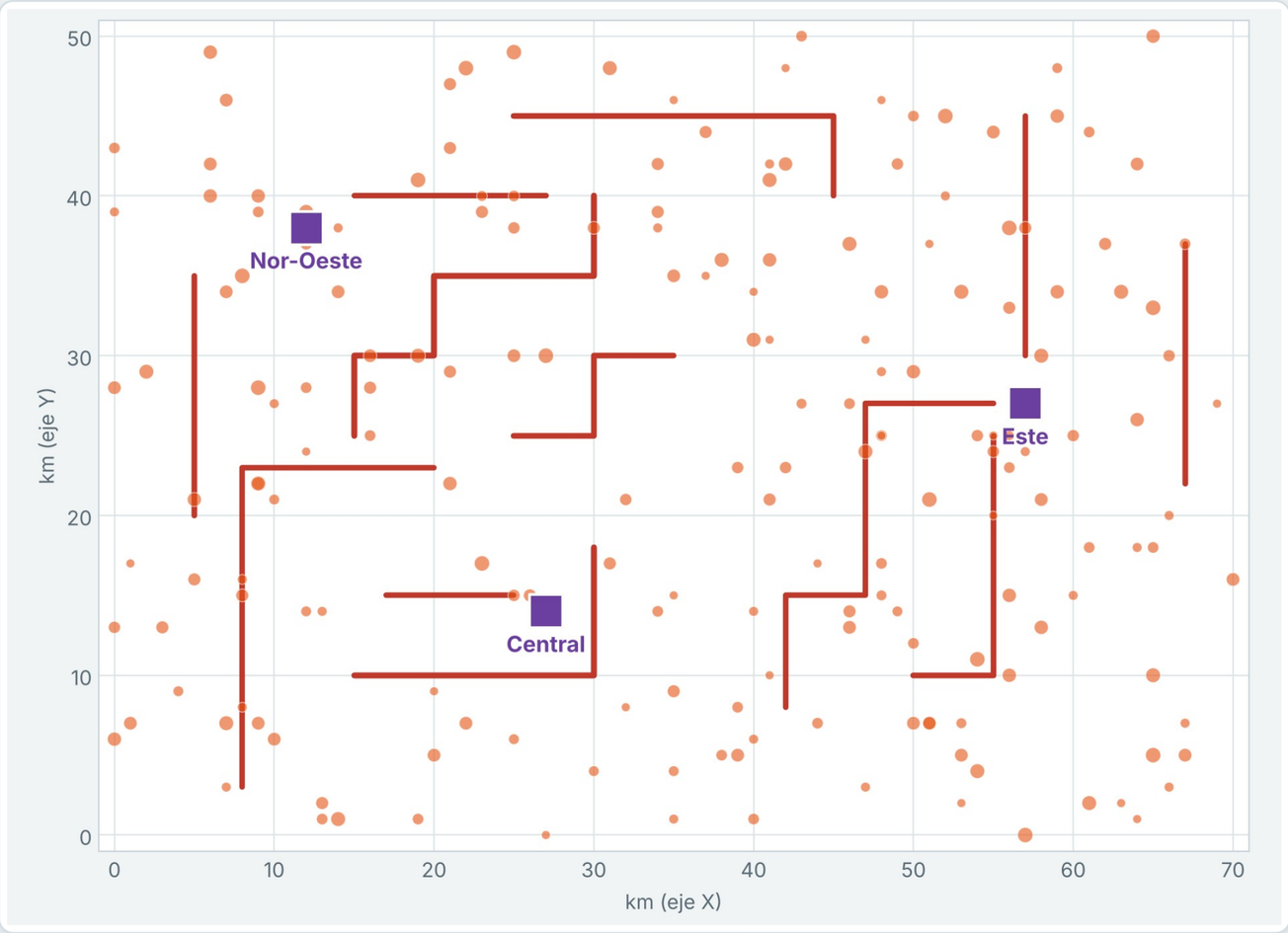
PaqRap: Sistema de Entrega Rápida

Cómo el software resuelve el problema de las entregas y cómo está construido

Equipo 2B · Horario 0982 · Proyecto de Diseño y Desarrollo de Software (1INF54)

Leslie Díaz · Ariana Silva · Axel Arista · Melvin Custodio

El problema: entregar a tiempo con poca flota



Un día real (01/09/2026): 187 pedidos y 23 bloqueos nuevos

Una ciudad de 70 × 50 km

Calles rectas de doble sentido, nodos cada 1 km

37 unidades de 3 tipos

10 autos · 15 motos · 12 bicicletas

Plazos de 4 a 36 horas

Cada pedido tiene su fecha límite

Imprevistos en plena operación

Bloqueos de calles, averías y mantenimiento

La demanda real: septiembre de 2026

5 000

pedidos en el mes

27 494

productos a entregar

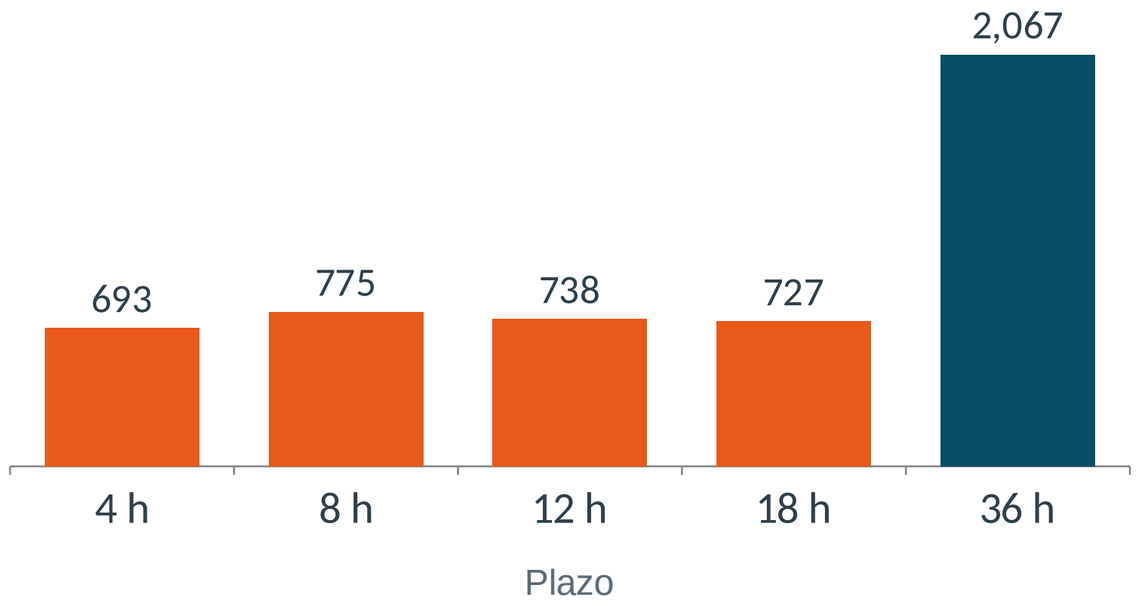
558

bloqueos de calles

64–217

pedidos por día

Pedidos según su plazo de entrega



Casi 6 de cada 10 pedidos vencen en 18 h o menos

Nuestra solución: planificar, simular y visualizar

1 · Datos

Pedidos, bloqueos y mantenimiento oficiales, guardados en MySQL (AWS)

2 · Planificador

Asigna pedidos a unidades y arma las rutas: Vecino Más Cercano + Búsqueda Tabú

3 · Simulador

Avanza el reloj, aplica bloqueos, averías y mantenimiento, y replanifica

4 · Visualizador

Mapa en vivo con rutas, indicadores y averías, desde cualquier dispositivo

Resultado: cada pedido llega dentro de su plazo, con el menor costo de recorrido posible

Tres escenarios para evaluar la operación



Operación día a día

Tiempo real (1×)

Una jornada con pedidos reales; se acelera 60× para demostrar (≈ 24 min). Permite averías en vivo.



Simulación de 5 días

5 días en 30–60 min

Cinco jornadas seguidas para comparar el comportamiento del periodo (40 min por defecto).



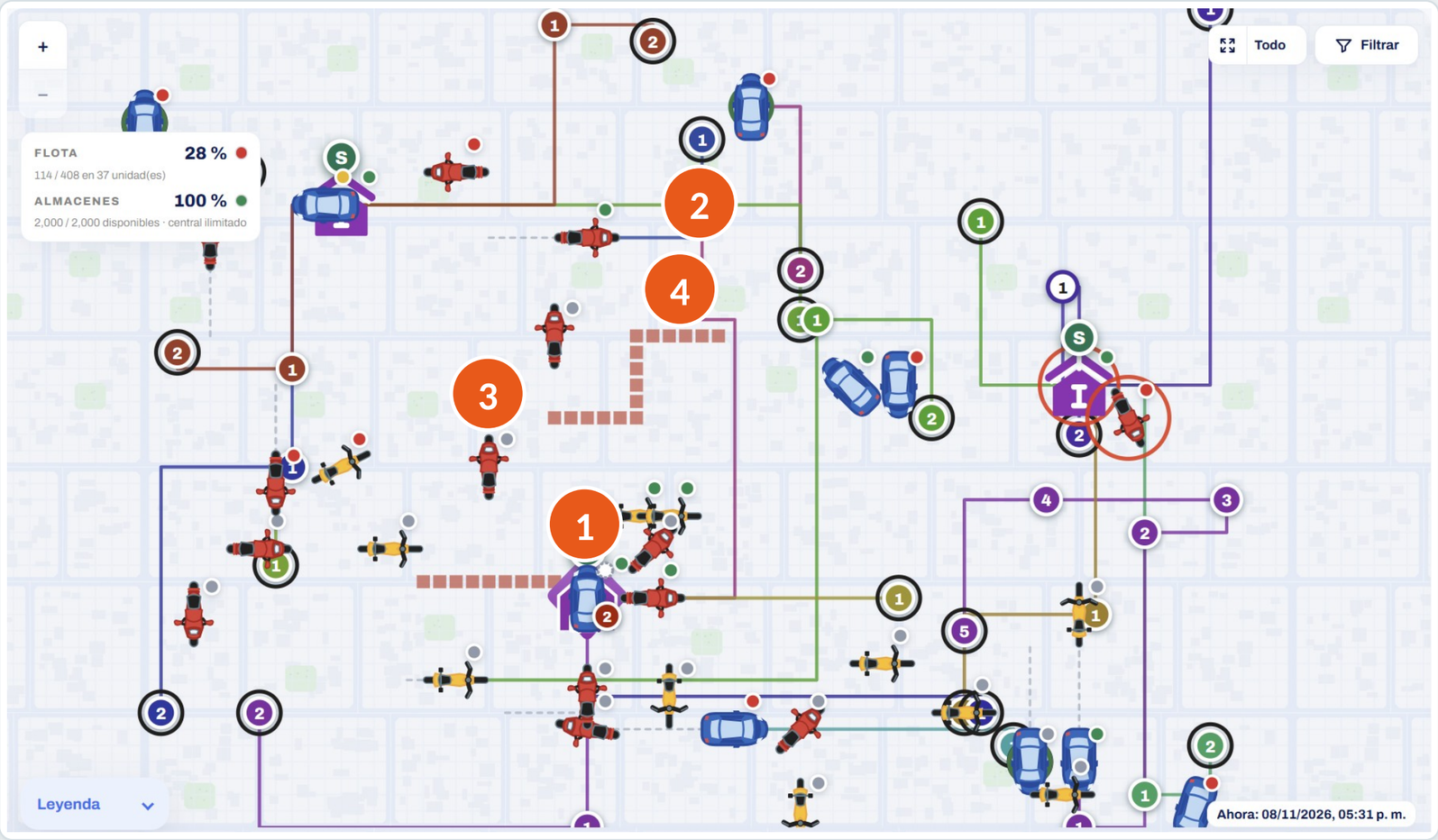
Colapso logístico

Hasta el 1.er vencido

Corre hasta que un pedido vence sin entregarse: hasta 30 días, como máximo 1 h real.

El mismo planificador y el mismo simulador corren los tres escenarios; solo cambia el ritmo

Funcionalidad 1: mapa de la operación en vivo



Corrida del 01/09/2026 con los datos oficiales (5 000 pedidos en el mes)

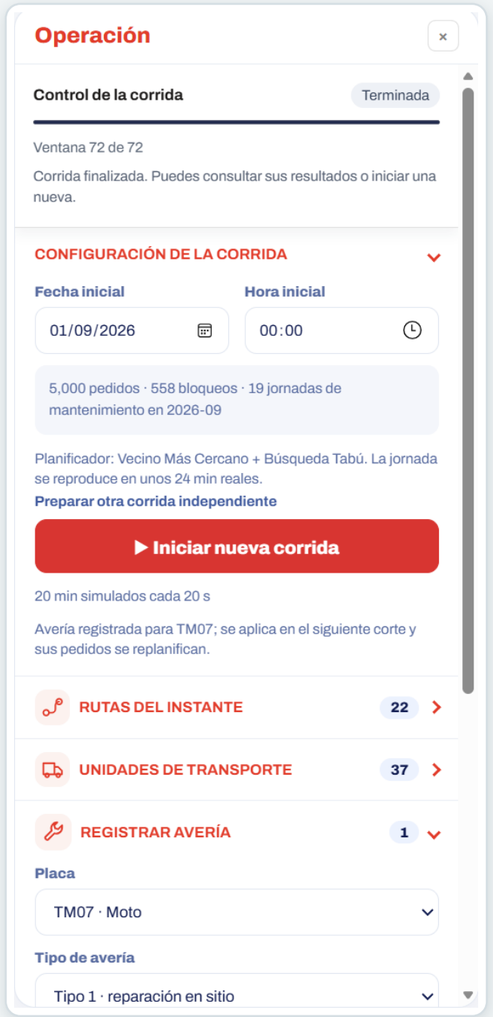
1 Almacenes
Central e intermedios, con su stock

2 Rutas
Una por unidad, con paradas numeradas

3 Unidades
Autos, motos y bicicletas en movimiento

4 Bloqueos
Solo mientras están vigentes

Funcionalidad 2: control de la corrida



Iniciar, pausar y detener

Valen para todos los dispositivos conectados a la misma corrida

Fecha y hora de inicio

El sistema informa cuántos pedidos, bloqueos y jornadas de mantenimiento tiene el mes

Rutas del instante

22 rutas en la captura, con distancia y costo en soles por ruta

Unidades de transporte

Las 37 unidades por tipo y estado

Funcionalidad 3: averías de la flota



Prueba local: el pedido 4-D-001 pasó de TA01 (averiada) a TA06

TIPO 1
Menor

Unidad fuera 2 horas

TIPO 2
Intermedia

Fuera hasta el fin del siguiente turno (máx. 4 h en el lugar)

TIPO 3
Mayor

Fuera al menos 2 días; vuelve en el turno de 15:00 a 23:00

Tipos 2 y 3: la unidad y sus paquetes van al almacén central (30 min de trasvase)

Funcionalidad 4: seguimiento y reproducción

154	31	1	0
Entregados	En ruta	Planificados	Por planificar



Corrida del 01/09/2026 a 60×: reproducción 91/91, duración 24 min 01 s

Historial

Se revisa cualquier instante con el deslizador o «En vivo»

Varios dispositivos

Otro equipo toma la corrida activa al abrir la página, o con el ID en la URL

Pedidos

Lista con la situación de cada pedido y su seguimiento

Requisitos: qué debe cumplir el software

100
exigencias del software

15
casos de uso

3
escenarios de evaluación

2
metaheurísticas en Java

Del usuario

- Elegir escenario, fecha y hora
- Iniciar, pausar y detener la corrida
- Registrar averías en vivo
- Ver mapa, rutas, pedidos e indicadores

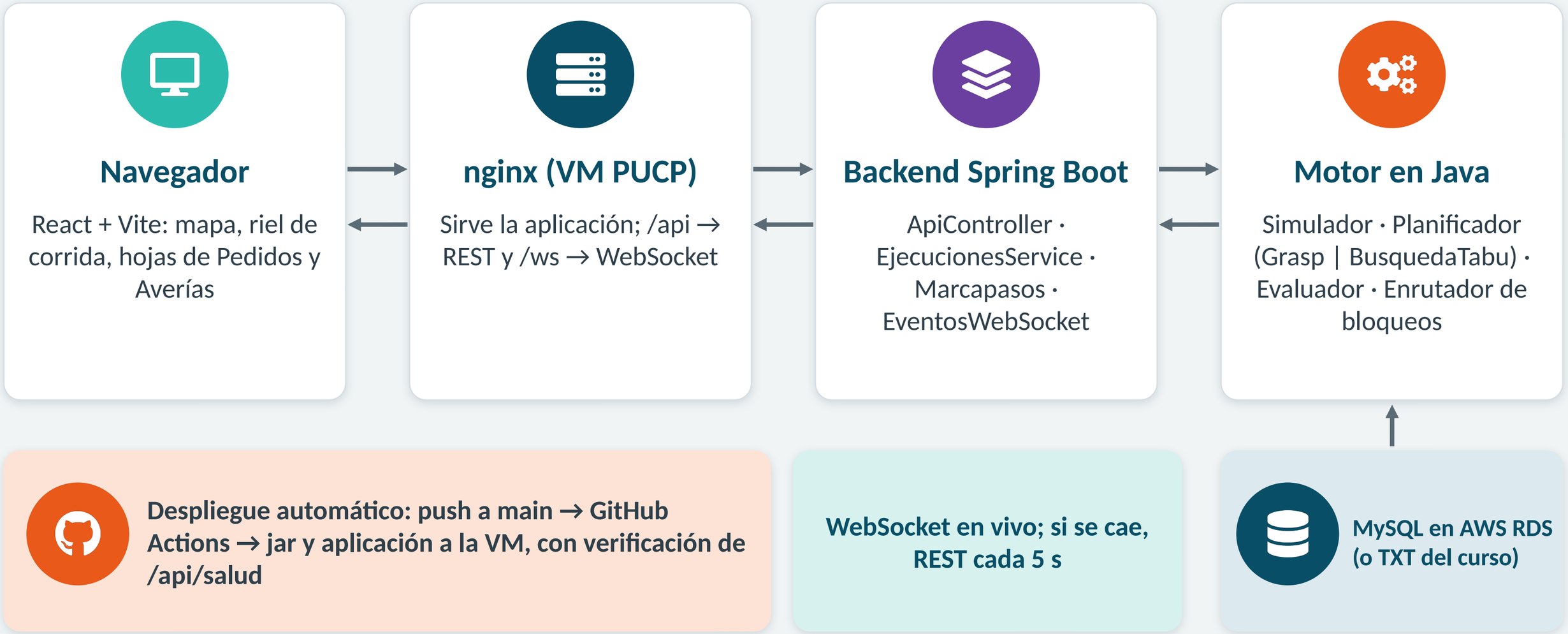
Del negocio

- Plazos de 4 a 36 horas
- 3 tipos de avería
- Bloqueos por horario
- Mantenimiento preventivo por día

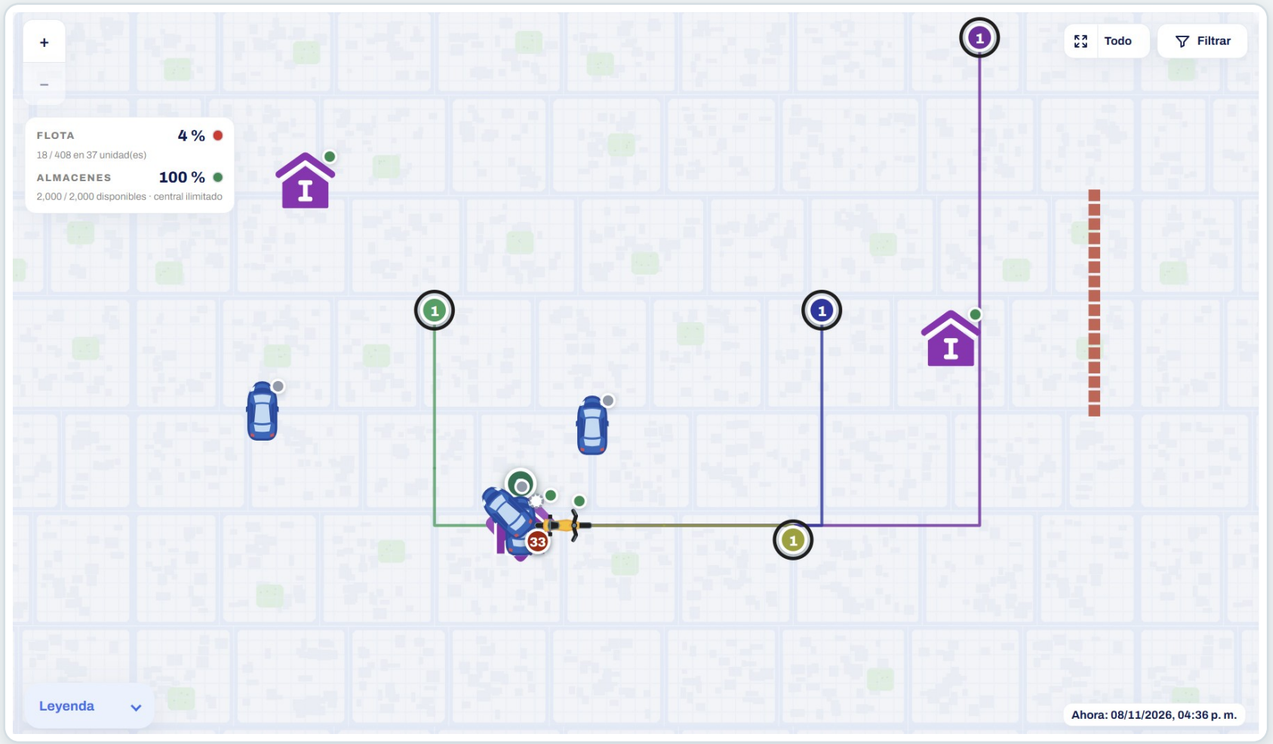
Técnicos

- Planificador con 2 metaheurísticas
- 5 días en 30 a 60 min
- VM de la PUCP y BD en AWS
- Acceso desde varios dispositivos

Arquitectura de PaqRap



Gestor de visualización en el mapa



Si el WebSocket se cae, el navegador consulta por REST cada 5 s y la corrida no se pierde

1. El servidor publica una ventana cada Sa (20 s)
2. El WebSocket envía rutas, paradas, hitos y stock
3. Un reloj anclado mueve cada unidad, sin saltar hacia adelante
4. El mapa dibuja el plan vigente de ese instante

Varias unidades en un nodo: un contador evita superponerlas

Planificador de rutas para los 3 escenarios

Escenario	Horizonte	Ventana real (Sa)	Operación por ventana (Sc)	Duración real
Día a día	1 día	20 s	1× (o 60×)	24 h (≈ 24 min a 60×)
5 días	5 días	20 s	≈ 1 h	30 a 60 min
Colapso	1 a 60 días (30)	10 s	2 h	Hasta el 1.er vencido

Una sola planificación se vuelve a calcular cuando algo cambia:



Cada 30 min



Llega un pedido



Empieza o termina un bloqueo



Se registra una avería

Planificador.planificar(estado de la operación) → rutas por unidad + pedidos no asignados

Cómo se simula y qué limitaciones aceptamos

Cómo se simula

El reloj simulado avanza hasta el siguiente corte

El simulador aplica bloqueos, averías y mantenimiento

El planificador calcula el plan

El marcapasos publica la ventana y el ciclo se repite

La avería registrada entra al cálculo al final de la ventana visible

Limitaciones aceptadas

- Corridas en memoria: se pierden al reiniciar (3 simultáneas + 2 en cola)
- Sin autenticación: acceso controlado en la VM
- Trazo esquemático de la retícula, no GPS
- Pedidos nuevos solo desde los archivos del curso

Algoritmo: Vecino Más Cercano + Búsqueda Tabú

Estado de la operación

Pedidos pendientes, unidades, stock y bloqueos vigentes

1 · Constructor Vecino Más Cercano

Atiende primero el pedido de menor holgura; desempata por mayor arrepentimiento

2 · Búsqueda Tabú

Mejora el plan explorando vecinos, sin repetir movimientos recientes

Plan resultante

Rutas por unidad y pedidos no asignados

Evaluador común: valida capacidad, stock, bloqueos y plazos, y calcula costo y penalizaciones

Semilla fija: resultados reproducibles

Se recalcula en cada corte de la simulación

Cómo explora la Búsqueda Tabú

1 · Genera una muestra de 40 vecinos

Se repite 120 iteraciones

2 · Elige el mejor movimiento no tabú

3 · Lo aplica y lo registra

Memoria tabú

recuerda los últimos 8 movimientos

Aspiración: se rompe la prohibición si el movimiento mejora la mejor solución conocida

Movimientos del vecindario

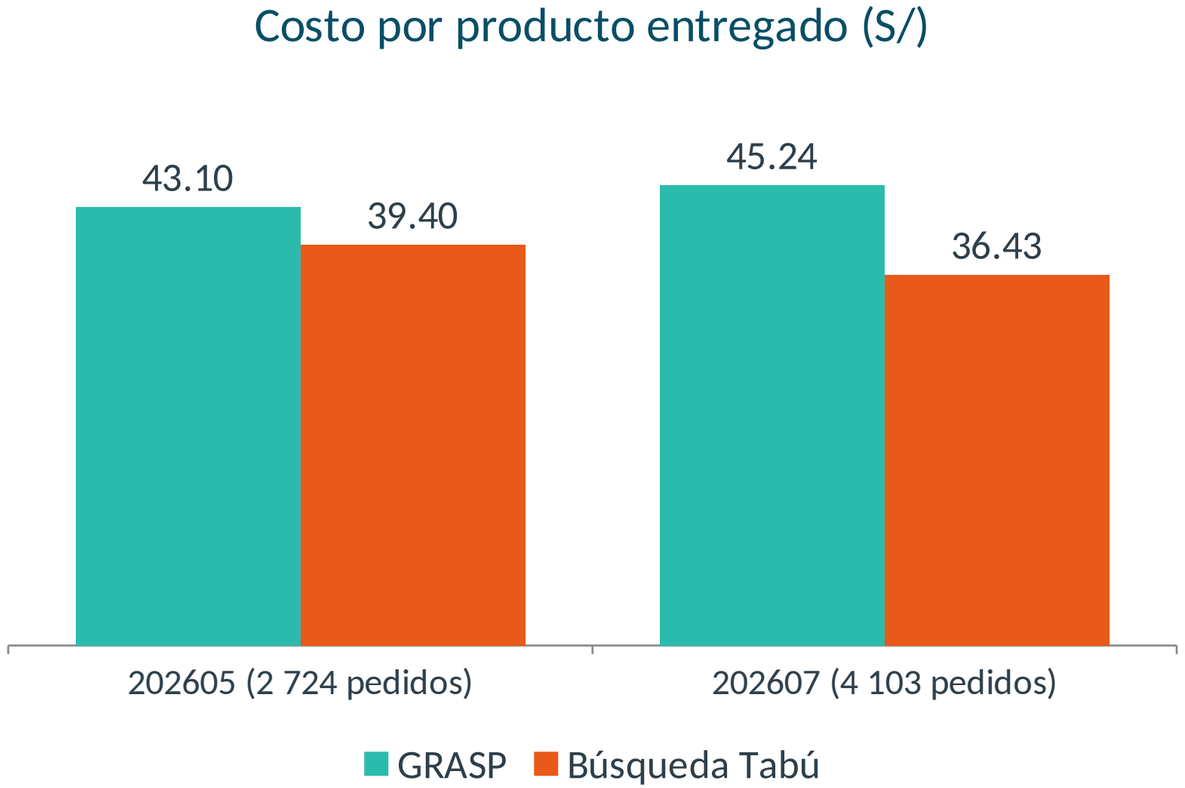
- Trasladar un pedido
- Intercambiar dos pedidos
- 2-opt: invertir un tramo
- Cambiar de vehículo
- Cambiar de almacén

Parámetros de la versión desplegada

Iteraciones	Tenencia	Muestra	Penalidad
120	8	40	80

Semilla fija: la misma entrada da el mismo plan

Experimento: Tabú gana con alta demanda



−19 %

costo por producto (202607)

−20 %

km recorridos: 119 mil frente a 148 mil

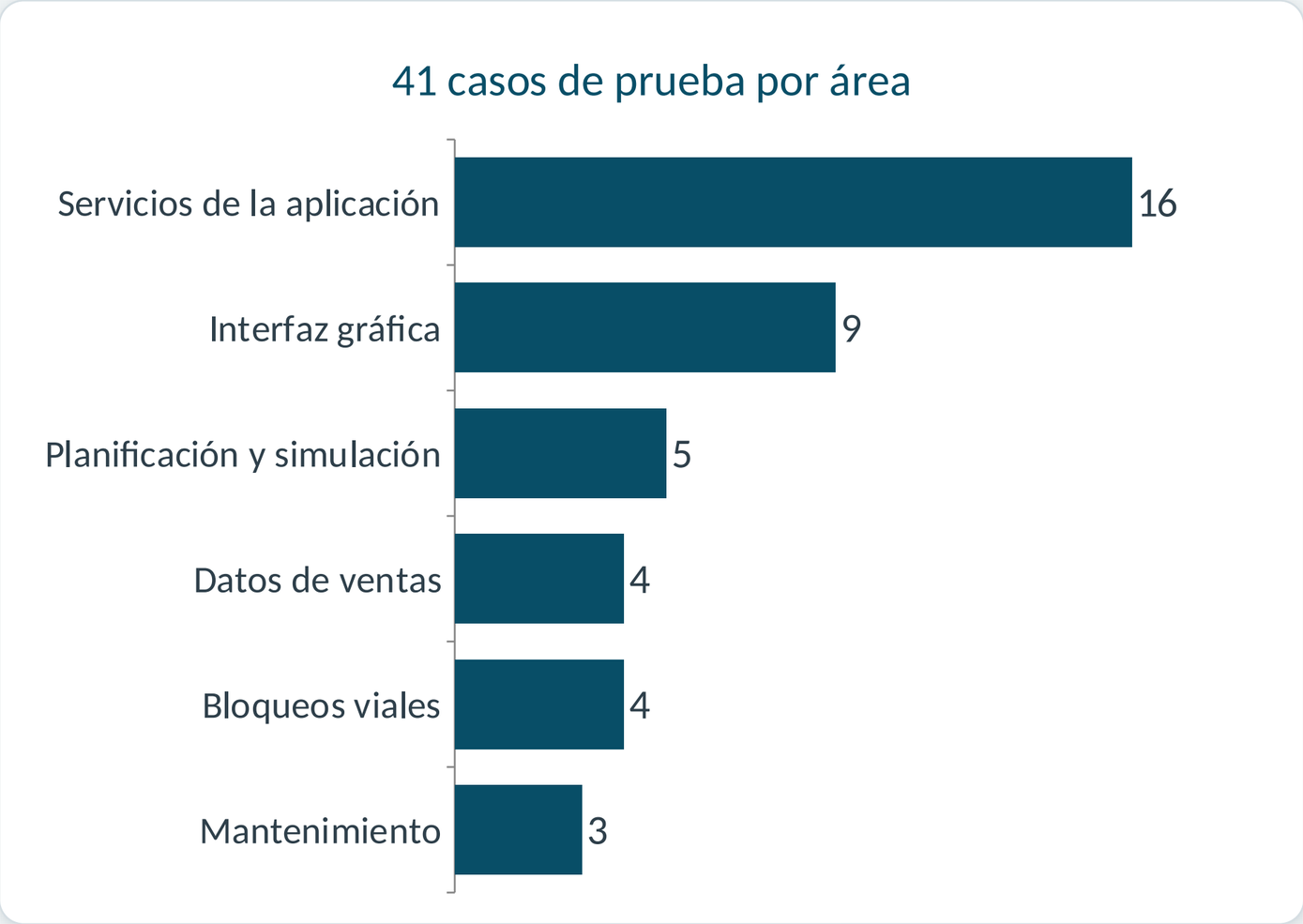
−38 %

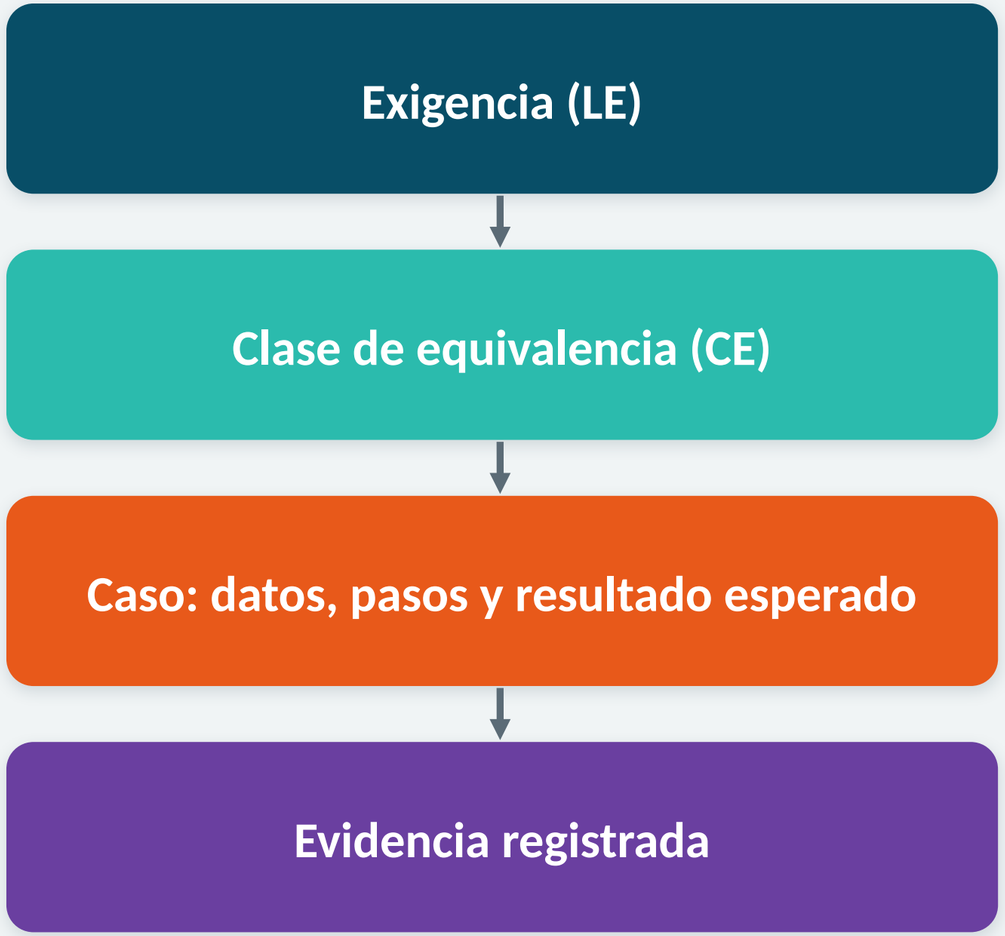
tiempo de cálculo: 197 s frente a 319 s

80 ejecuciones: 4 periodos reales × 10 semillas × 2 algoritmos

Con poca demanda (202601 y 202603) GRASP fue más barato y rápido: la decisión depende del volumen

Catálogo de pruebas de aceptación





Pruebas realizadas y resultados

168
pruebas automatizadas aprobadas

- 137 del frontend
- 31 del motor y backend (al 18/10)

5
casos ejecutados con evidencia

- Avería tipo 2: 2 pedidos replanificados
- PC y celular ven la misma corrida
- 5 días: sin colapso, 75 pedidos
- Colapso: se detiene a las 09:15
- Día real 01/09: 154 entregas

41
casos por ejecutar en la VM

- Cancelar una corrida de punta a punta
- Medir el rendimiento en la VM

Los 41 casos están diseñados; ejecutarlos formalmente en la VM es el siguiente paso

Conclusiones



Resuelve el problema

Planifica, replanifica ante imprevistos y muestra la operación en vivo

Con respaldo

Algoritmo evaluado en 80 ejecuciones y 41 casos de prueba diseñados

Siguientes pasos

Ejecutar el catálogo completo en la VM, agregar autenticación y persistir corridas

Gracias. ¿Preguntas?